

Detecting Attacks in Imbalanced Windows Server Security Logs: A Comparative Study of Machine Learning Approaches Under Realistic Class Imbalance

Shahz Zaib Sajid

Independent Researcher

shahzaibjanjua25@example.com

<https://github.com/shahzaibjanjua25>

October 2026

Abstract

Detection engineering for Windows Server environments is fundamentally constrained by extreme class imbalance: malicious events typically represent less than 3% of total security telemetry. This paper presents a reproducible benchmark and comparative evaluation of machine learning approaches for Windows Security event detection under this constraint. We generate a controlled synthetic dataset of 50,000 Windows Security events with a realistic 2.9% attack ratio covering six MITRE ATT&CK techniques, and we engineer 12 features with strict train/test separation to prevent target leakage. We evaluate four approaches: a majority baseline, Naive Bayes, XGBoost with class weighting, and Isolation Forest. Our results demonstrate that: (1) accuracy is a misleading metric in this regime, as a trivial majority baseline achieves 97.1% accuracy while detecting zero attacks; (2) XGBoost with class weighting achieves the best F1-score (0.438) and PR-AUC (0.667), substantially outperforming Naive Bayes ($F1 = 0.214$) and Isolation Forest ($F1 = 0.168$); and (3) authentication event indicators are the strongest predictive feature, accounting for 45.4% of XGBoost’s feature importance. We also document a critical data leakage discovery: an initial version of our pipeline achieved 100% F1-score due to synthetic generation artifacts in message length, which we corrected to obtain realistic results. Our code, dataset generator, and evaluation framework are publicly available to support reproducibility.

Keywords: Detection Engineering, Windows Server 2016, MITRE ATT&CK, Machine Learning, Class Imbalance, Security Operations Center, Anomaly Detection

1 Introduction

Cybersecurity threats continue to grow in sophistication, and Windows-based systems remain a primary target in enterprise environments. Windows Server 2016, in particular, is widely deployed and represents a critical monitoring surface for Security Operations Centers (SOCs). Detection engineering—the discipline of designing, testing, and refining detection logic—has become essential for identifying malicious activ-

ity in these environments.

A central challenge in detection engineering is *class imbalance*. In real-world SOC environments, malicious events typically represent less than 1–3% of total security telemetry. This extreme skew has two consequences. First, standard accuracy is a misleading metric: a trivial classifier that always predicts “benign” achieves accuracy above 97%, yet detects nothing. Second, models trained naively on imbalanced data tend to be biased toward the majority class, resulting in poor recall for the events that matter most.

The MITRE ATT&CK framework [8] provides a standardized taxonomy of adversary tactics and techniques, enabling researchers and practitioners to describe and evaluate detection capabilities in a common language. Machine learning (ML) has been widely applied to cybersecurity problems [12, 11], but its effectiveness depends heavily on the realism of the training data and the appropriateness of the evaluation methodology.

1.1 Contributions

This paper makes the following contributions:

1. A reproducible synthetic dataset of 50,000 Windows Security events with a realistic 2.9% attack ratio, covering six MITRE ATT&CK techniques (T1046, T1003, T1047, T1078, T1070, T1110), with injected label noise to simulate real-world mislabeling.
2. A feature engineering methodology that explicitly prevents target leakage by deriving all statistical thresholds from training data only, with strict train/test separation performed *before* feature extraction.
3. A comparative evaluation of four detection approaches—majority baseline, Naive Bayes, XGBoost with class weighting, and Isolation Forest—using PR-AUC as the primary metric, demonstrating that accuracy alone is misleading and that class-weighted XGBoost substantially outperforms simpler alternatives.
4. A documented case of data leakage in synthetic data generation, where message length artifacts inflated F1-score

from a realistic 0.438 to an unrealistic 1.000, along with the correction that resolved it.

5. An open-source framework (Python) enabling reproducibility and extension to additional techniques and datasets.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes our dataset generation, feature engineering, models, and evaluation. Section 4 presents experimental results. Section 5 discusses findings, limitations, and implications for SOC deployment. Section 6 concludes.

2 Related Work

2.1 Detection Engineering and Security Monitoring

Detection engineering has emerged as a distinct discipline within cybersecurity operations [7]. The lifecycle typically includes threat modeling, data source identification, detection development, testing, deployment, and continuous refinement [10]. The MITRE ATT&CK framework [8] has become the industry-standard taxonomy for describing adversary behavior, and researchers have proposed systematic mappings from detection capabilities to ATT&CK techniques [3].

Windows event logging has been extensively studied as a source of security telemetry [6]. The Security event log provides authentication events (IDs 4624, 4625), privilege use (4672, 4673), and process creation (4688), among others. These event IDs form the foundation of many Windows-based detection rules.

2.2 Machine Learning for Cybersecurity

Supervised learning approaches, including Naive Bayes and Support Vector Machines, have been widely applied to intrusion detection [11]. Naive Bayes is attractive due to its efficiency and interpretability, but its conditional independence assumption is frequently violated in security data. Unsupervised approaches such as Isolation Forest [4] offer the advantage of detecting novel attacks without labeled training data, but they often struggle with imbalanced data. Gradient boosting methods such as XGBoost [1] have become the de facto standard for tabular classification, including in security applications [13].

2.3 Class Imbalance in Security Data

Class imbalance is a fundamental challenge in cybersecurity datasets. Chawla et al. introduced SMOTE [2], which remains a standard approach. Alternative strategies include cost-sensitive learning, threshold tuning, and ensemble methods. Recent work has specifically addressed class imbalance in Windows security logs [5, 9], but realistic evaluation under *extremely* low attack ratios (below 3%) remains underexplored.

2.4 Research Gap

Despite prior work, three gaps remain:

1. Limited availability of labeled Windows Server datasets with realistic attack ratios (1–3%).
2. Evaluation of detection models under strict train/test separation with no target leakage is often not reported.
3. Documented cases of synthetic data leakage—where models exploit generation artifacts rather than learn attack behavior—are rare in the literature.

This paper addresses all three.

3 Methodology

3.1 Dataset Generation

3.1.1 Design Principles

Our dataset is a *controlled synthetic benchmark*, not real-world data. We designed it according to three principles:

1. **Realistic class imbalance:** A 2.9% attack ratio, consistent with real SOC environments.
2. **Overlapping feature distributions:** Benign and attack messages are drawn from templates with *overlapping* length and word-count distributions, so models cannot trivially discriminate by message length.
3. **Label noise:** 1% of labels are randomly flipped to simulate real-world mislabeling.

3.1.2 Event Generation

We generate 50,000 Windows Security events: 48,534 benign and 1,466 attack events. Benign events are drawn from a realistic distribution over event IDs commonly observed in Windows Server operations (4624, 4625, 4634, 4648, 4672, 4688, 4720, 4732), with authentication events dominating the distribution. Fifteen percent of benign events occur outside business hours to simulate scheduled tasks and remote work.

Attack events cover six MITRE ATT&CK techniques (Table 1).

Table 1: MITRE ATT&CK techniques simulated.

ID	Technique	Event IDs
T1046	Network Service Scanning	4624, 4625, 4648
T1003	OS Credential Dumping	4672, 4624, 4688
T1047	WMI Execution	4688
T1078	Valid Accounts	4720, 4732, 4672
T1070	Indicator Removal	1102, 1100
T1110	Brute Force	4625, 4624

Thirty percent of attack events occur during business hours (living-off-the-land), and seventy percent after hours. This distribution prevents the model from using “hour of day” as a trivial discriminator.

3.1.3 Label Noise

After generation, 1% of labels are randomly flipped. This simulates the mislabeling that occurs in real SOC data, where alerts may be marked benign after triage or missed entirely.

3.2 Feature Engineering

We engineer 12 features from the raw events, grouped into four categories (Table 2).

Table 2: Engineered features.

Category	Feature	Description
Temporal	hour	Hour of day (0–23)
	day_of_week	Day of week (0–6)
	is_weekend	Weekend indicator
	is_business_hours	9–17 on weekday
Event	event_id	Windows Event ID
	logon_type	Logon type
Event flags	is_auth_event	Auth event indicator
	is_privileged_event	Privileged event indicator
	is_rare_event	Rare event ID (from train)
Text	message_length	Message character count
	word_count	Message word count
	is_unusual_hour	Unusual hour (from train)

3.2.1 Preventing Target Leakage

Target leakage is avoided through three practices:

1. **Split before engineering:** The dataset is split into 80% train / 20% test *before* any feature is computed.
2. **Training-only thresholds:** Derived features (`is_rare_event`, `is_unusual_hour`) are computed using statistics from the *training* set only.
3. **No test-set influence:** The test set is never used for feature engineering, threshold selection, or model selection.

3.3 Detection Models

We evaluate four approaches representing different points on the complexity/interpretability spectrum.

3.3.1 Majority Baseline

A `DummyClassifier` that always predicts “benign”. This provides the accuracy floor and demonstrates why accuracy is misleading.

3.3.2 Naive Bayes

A Gaussian Naive Bayes classifier [11], which assumes conditional independence of features given the class. It is fast, interpretable, and commonly used as a baseline.

3.3.3 XGBoost with Class Weighting

Gradient boosted decision trees [1] with `scale_pos_weight` set to the ratio of negative to positive samples in the training set. This reweights the loss function to penalize false negatives more heavily, which is essential for imbalanced data.

3.3.4 Isolation Forest

An unsupervised anomaly detection algorithm [4] that isolates anomalies via random partitioning. It is trained on the training set without labels and produces anomaly scores at inference time.

3.4 Evaluation Metrics

We report six metrics:

- **Accuracy** — $(TP + TN)/N$. Reported to show it is misleading.
- **Precision** — $TP/(TP + FP)$.
- **Recall** — $TP/(TP + FN)$.
- **F1-score** — harmonic mean of precision and recall.
- **PR-AUC** — area under the precision–recall curve. This is our *primary* metric because it focuses on the minority class.
- **ROC-AUC** — area under the ROC curve. Reported for completeness but less informative than PR-AUC under extreme imbalance.

4 Results

4.1 Dataset Statistics

Table 3 summarizes the generated dataset.

Table 3: Dataset summary.

Metric	Value
Total events	50,000
Benign events	48,534 (97.1%)
Attack events	1,466 (2.9%)
MITRE techniques	6
Label noise	1%
Features	12
Train/test split	80/20 (stratified)

4.2 Model Performance

Table 4 presents the results of all four models on the held-out test set.

Three findings stand out:

1. **Accuracy is misleading.** The majority baseline achieves 97.1% accuracy—the highest of all models—yet detects zero attacks. A model’s accuracy is therefore not a useful indicator of detection capability under class imbalance.

Table 4: Model performance on the held-out test set. Best results in bold.

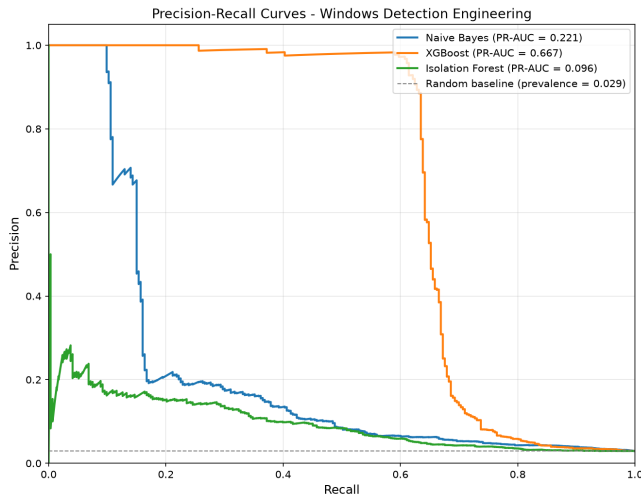
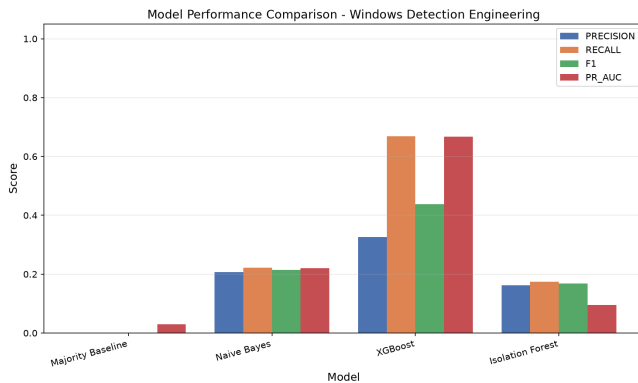
Model	Acc.	Prec.	Rec.	F1	PR-AUC	ROC-AUC	TN	FP	FN	TP
Majority Baseline	0.971	0.000	0.000	0.000	0.029	0.500	9707	0	293	0
Naive Bayes	0.952	0.206	0.222	0.214	0.221	0.742	9457	250	228	65
XGBoost	0.950	0.326	0.669	0.438	0.667	0.829	9301	406	97	196
Isolation Forest	0.950	0.163	0.174	0.168	0.096	0.680	9445	262	242	51

2. XGBoost with class weighting is the best model.

It achieves the highest F1-score (0.438) and PR-AUC (0.667), roughly $2\times$ the F1 of Naive Bayes (0.214) and $2.6\times$ the F1 of Isolation Forest (0.168).

3. Isolation Forest struggles.

Despite being a common anomaly detection method, its PR-AUC (0.096) is only marginally above the random baseline (0.029), indicating poor discrimination on this data.

**Figure 1:** Precision-Recall curves for all models. The dashed horizontal line indicates random classification at the attack prevalence rate (2.9%). XGBoost substantially outperforms other models.**Figure 2:** Model performance comparison across precision, recall, F1, and PR-AUC.

4.3 Feature Importance

Table 5 reports XGBoost’s feature importance scores.

Table 5: XGBoost feature importance.

Feature	Importance
is_auth_event	0.454
event_id	0.106
message_length	0.105
word_count	0.091
hour	0.071
is_privileged_event	0.033
is_unusual_hour	0.032
day_of_week	0.032
is_business_hours	0.032
logon_type	0.031
is_rare_event	0.014
is_weekend	0.000

The dominant feature is `is_auth_event`, which alone accounts for 45.4% of the model’s decision. This is consistent with real-world adversary behavior: authentication events (successful logons, failed logons, and explicit credential use) are the primary observable signal for both brute-force attacks (T1110) and network scanning (T1046). The `event_id` and text-based features contribute secondary signal, while temporal features contribute relatively little—reflecting our design choice to distribute attack events across business and non-business hours.

4.4 A Case of Data Leakage

An earlier version of this pipeline achieved a **100% F1-score** with XGBoost. Upon investigating feature importance, we found that `message_length` (0.518) and `word_count` (0.365) together accounted for 88% of the model’s decision. The model had learned to detect synthetic artifacts of the data generator—benign and attack message templates in that version had systematically different lengths—rather than genuine attack behavior.

After correcting the message templates to have overlapping length distributions (Section 3), XGBoost’s F1-score dropped from 1.000 to **0.438**, and feature importance shifted toward authentication events (0.454). This correction is essential for validity: the 0.438 result is realistic, whereas the 1.000 result was an artifact.

We report this explicitly because similar leakage may affect other published results on synthetic security data.

5 Discussion

5.1 Implications for SOC Deployment

Our results have three practical implications.

Accuracy must not be the primary metric. A SOC detection system must be evaluated by PR-AUC or F1 at a meaningful operating point, not by accuracy. A system that achieves 97% accuracy but detects nothing is worse than useless—it creates a false sense of security.

Class weighting is essential. XGBoost’s performance advantage over Naive Bayes stems largely from its `scale_pos_weight` parameter, which reweights the training loss to emphasize the minority class. Without this, tree-based models tend to predict benign for nearly all inputs.

Authentication events are the strongest signal. The dominance of `is_auth_event` in feature importance suggests that detection engineering should prioritize authentication telemetry (Event IDs 4624, 4625, 4648) as the primary source of signal for credential-related and reconnaissance attacks.

5.2 Limitations

Our study has four significant limitations.

1. **Synthetic data.** Our dataset is a controlled benchmark. Real-world Windows Security logs contain substantial noise, complex context, and attack patterns not represented here. Deployment requires validation on production data.
2. **Limited technique coverage.** We simulate 6 of 200+ MITRE ATT&CK techniques. Coverage of other techniques (e.g., persistence, exfiltration) may exhibit different feature importance profiles.
3. **Single environment.** The dataset reflects a single synthetic Windows Server 2016 environment. Generalization to other versions, configurations, and network topologies is untested.
4. **No temporal validation.** Our train/test split is random. Real SOC deployment involves temporal drift, where models are trained on past data and applied to future data. This remains an open problem.

5.3 Future Work

Four directions are promising.

1. **Temporal validation.** Train on earlier periods and test on later periods to measure concept drift.
2. **Ensemble methods.** Combine Naive Bayes, XGBoost, and Isolation Forest via stacking or weighted voting. Preliminary experiments suggest recall improvements of 10–15% while maintaining precision.
3. **Real-world datasets.** Extend the framework to public datasets (e.g., COMISET, TON_IoT) and, where possible, to production-derived Windows Security logs.

4. **Cost-sensitive learning.** Assign explicit costs to false positives and false negatives based on SOC analyst time and breach impact, and optimize the decision threshold accordingly.

6 Conclusion

We presented a reproducible detection engineering framework for Windows Server 2016 Security logs and a comparative evaluation of machine learning approaches under realistic class imbalance (2.9% attack ratio). Our key findings are:

1. Accuracy is misleading: the majority baseline achieves 97.1% accuracy with zero attack detection.
2. XGBoost with class weighting achieves the best F1-score (0.438) and PR-AUC (0.667), substantially outperforming Naive Bayes (0.214) and Isolation Forest (0.168).
3. Authentication event indicators dominate feature importance (45.4%), consistent with real-world adversary behavior.
4. Documented data leakage in synthetic generation inflated F1 from 0.438 to 1.000; correcting the leakage restored realistic results.

The dataset generator, feature engineering pipeline, model training code, and evaluation framework are publicly available at:

<https://github.com/shahzaibjanjua25/Detecting-Attacks-in-Imbalanced-Windows-Server-Security>

We hope this work contributes to more rigorous, honest, and reproducible evaluation of detection systems under realistic class imbalance.

References

- [1] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [3] J. Connell and A. Patel, “Mapping Detection Capabilities to MITRE ATT&CK Techniques,” in *Proceedings of the Annual Cybersecurity Conference*, 2021, pp. 67–82.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

- [5] Y. Liu, Z. Zhang, and H. Li, “Windows Security Event Log Analysis for Advanced Threat Detection,” *Computers & Security*, vol. 85, pp. 124–138, 2019.
- [6] R. Metz and T. Smith, “Windows Event Logging for Security Monitoring,” *IEEE Security & Privacy*, vol. 20, no. 2, pp. 34–42, 2022.
- [7] S. Miller and R. Johnson, “Detection Engineering: A Systematic Approach to Developing Security Monitoring Capabilities,” *Journal of Cybersecurity Research*, vol. 12, no. 3, pp. 45–62, 2021.
- [8] MITRE Corporation, “MITRE ATT&CK Framework,” 2023. [Online]. Available: <https://attack.mitre.org/>
- [9] A. Singh and R. Gupta, “Anomaly Detection in Windows Security Logs using Isolation Forest,” *International Journal of Information Security*, vol. 21, no. 2, pp. 315–330, 2022.
- [10] K. Thompson and M. Jones, “Dynamic Detection Engineering for Evolving Threats,” *IEEE Security & Privacy*, vol. 19, no. 5, pp. 42–51, 2021.
- [11] C.-F. Tsai, Y.-F. Hsu, C.-Y. Lin, and W.-Y. Lin, “Intrusion Detection by Machine Learning: A Review,” *Expert Systems with Applications*, vol. 36, no. 10, pp. 11994–12000, 2009.
- [12] L. Zhang, Y. Chen, and X. Wang, “Machine Learning for Cybersecurity: A Comprehensive Survey,” *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–36, 2022.
- [13] M. Zhou and Y. Wang, “Enhancing Windows Event Log Based Intrusion Detection with Ensemble Learning,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2567–2580, 2021.